

Quality Concept & Software Quality Assurance (SQA)

Quality Concept – Meaning

- Degree to which software meets requirements & customer needs
 - Software should work correctly
 - Should meet user expectations
 - Should be reliable and efficient

Diagram: Quality Concept Model

- User Needs (Expectations)
 - ↓
 - Requirements (Specifications)
 - ↓
 - Software Product
 - ↓
 - Quality = Meets Needs + Requirements

Two Views of Quality

- User View – Fitness for use, Easy to use, Customer satisfaction
- • Developer View – Conformance to requirements, No defects, Follows design

Software Quality Factors

- Correctness – Accurate results
- • Reliability – No failure
- • Efficiency – Minimum resources
- • Integrity – Security
- • Usability – Easy to operate
- • Maintainability – Easy to modify
- • Portability – Works on different platforms

Diagram: Software Quality Factors

- Software Quality
 - • → Correctness
 - • → Reliability
 - • → Efficiency
 - • → Integrity
 - • → Usability
 - • → Maintainability
 - • → Portability

Importance of Quality

- Reduces maintenance cost
- • Increases customer satisfaction
- • Improves company reputation
- • Reduces risk of failure

Software Quality Assurance (SQA) – Definition

- Planned & systematic activities
- • Ensures process & product conform to standards
- • Focuses on process quality

Objectives of SQA

- Ensure standards are followed
- • Prevent defects early
- • Improve reliability
- • Monitor development process
- • Ensure documentation

Objectives of SQA – With Examples

- Compliance with ISO standards
- • Clarify ambiguous requirements
- • Ensure backup & recovery testing
- • Stop process if phase not approved
- • Maintain SRS, Test Plan, User Manual

SQA Activities

- Quality Planning
- • Process Monitoring
- • Software Reviews
- • Audits
- • Testing Coordination
- • Configuration Management
- • Metrics Collection
- • Risk Management

Quality Planning

- Define standards
- • Select tools
- • Prepare SQA plan
- • Example: Coding standards & testing strategy

Process Monitoring

- Monitor SDLC phases
- • Ensure approvals taken
- • Prevent skipping of phases

Software Reviews

- Requirement Review
- • Design Review
- • Code Review
- • Remove ambiguity (e.g., 2 sec response time)

Audits

- Check documentation
- • Verify compliance with standards
- • Follow ISO guidelines

Testing Coordination

- Verify test plans
- • Check test cases
- • Ensure bugs logged properly

Configuration Management

- Control version updates
- • Track changes
- • Maintain backup copies

Metrics Collection

- Defect density
- • Code coverage
- • Review effectiveness
- • Improve testing if defect rate high

Risk Management

- Identify project risks
- • Example: Developer leaving project
- • Solution: Proper documentation & knowledge sharing

Diagram: SQA Activities Flow

- Quality Plan
- • ↓
- • Process Monitoring
- • ↓
- • Reviews & Audits
- • ↓
- • Testing & Metrics
- • ↓
- • Quality Product

Difference Between QA and QC

- QA – Process oriented, Prevents defects, Proactive
- • QC – Product oriented, Detects defects, Reactive

Diagram: QA vs QC

- Quality System
 - → Quality Assurance (Process Based, Prevents Defects)
 - → Quality Control (Product Based, Detects Defects)

Benefits of SQA

- Early defect detection
- • Reduced development cost
- • Better reliability
- • Improved productivity
- • Higher customer trust